FACULTY OF COMPUTING
UNIVERSITI TEKNOLOGI MALAYSIA
SEMESTER II · SESSION 2025 / 2026
SCSE2243 · SECTION 01 · APPLICATION DEVELOPMENT PROJECT I

FINAL REPORT

UTM BORROW

A Campus-Wide Circular Resource-Sharing Platform

PREPARED BY — GROUP ALPHA

NAME	MATRIC NO.	SUBSYSTEM OWNED
MOHAMMED MOHSEN MOHAMMED ALSAKKAF	A23CS4026	Transaction & Handover
MUAZ IBNE AHMED	A23CS4062	Resource Catalog & Discovery
AHMAT MAHAMAT MOURDJI MOUSTAPHA	A24CS4053	Identity & User Management

PREPARED FOR: Dr. Mohd Yazid Bin Idris
SUBMISSION DATE: 10 July 2026
REPOSITORY: github.com/Alsakkaf-dev/UTMBorrow

TABLE OF CONTENTS

*(In Microsoft Word, right-click the table above and choose **Update Field → Update entire table** to refresh page numbers.)*

1.0 INTRODUCTION TO APPLICATION

1.1 Introduction

UTM Borrow is a campus-wide circular resource-sharing platform for the Universiti Teknologi Malaysia (UTM) community. It gives verified students and staff one trusted place to lend and borrow the academic resources they only need for a short time — laboratory tools, engineering equipment, textbooks, and formal event clothing — instead of repeatedly buying expensive items that sit unused after a single semester. Every loan is governed by a strict state machine, and each physical handover and return is proven with a cryptographically signed QR code, so both parties remain accountable.

By converting a fragmented, buy-and-discard habit into a managed borrowing cycle, the platform lowers costs for students and advances UTM's Smart and Green Campus agenda and UN Sustainable Development Goal 12 (Responsible Consumption and Production). This final report documents the delivered system: its requirements, its as-built architecture, and a use-case-level account of exactly what was implemented across the three subsystems and nine modules that make up the application.

1.2 Problem Statement

UTM students routinely spend on specialised, single-use items — lab coats, scientific calculators, drawing sets, presentation attire — that lose all value after one semester. In the absence of a centralised campus platform, borrowing happens through unverified WhatsApp and Telegram groups that carry no identity checks, no reputation, and no record of who holds what. Four concrete problems follow from this gap:

- **Financial drain from single-use purchases.** Students, especially those from lower-income backgrounds, repeatedly buy costly items used only once.
- **No trust infrastructure.** Informal chat groups have no UTM-linked identity or reputation, exposing users to scams and theft by non-affiliates.
- **No accountability for returns and condition.** Without handover protocols or condition logs, items are returned late, damaged, or forgotten, and disputes cannot be resolved.
- **Wasted campus resources.** Redundant purchases every semester generate avoidable academic waste across the residential colleges.

1.3 The Proposed System

UTM Borrow resolves these problems through three integrated subsystems. The Identity & User Management subsystem restricts access to verified UTM email addresses and maintains a peer-driven Trust Score and a personal activity dashboard. The Resource Catalog & Discovery subsystem lets owners publish and manage listings and lets borrowers find items through structured category, condition, and location-based filtering. The Transaction & Handover subsystem manages the full borrowing lifecycle — Pending → Approved → Borrowed → Completed — with an atomic request lock, HMAC-SHA256 signed QR codes for tamper-proof handover and return, and a moderation pipeline for reporting and enforcement. Real-time updates are pushed to every client over Server-Sent Events, so notifications, catalog changes, and transaction status appear instantly without refreshing.

1.4 Technology and Tools Used

UTM Borrow is a responsive web application built on a three-tier client–server architecture. Every technology below reflects the system as actually built and deployed; the stack was chosen for security, real-time responsiveness, and a single maintainable codebase.

Layer	Technology	Role in UTM Borrow
Frontend	React 18 (CRA), Tailwind CSS, Framer Motion, Axios	Single-page UI for desktop and mobile browsers from one codebase; animated, token-authenticated calls to the API.
Backend	FastAPI (Python), asynchronous REST under /api	Request validation, authentication, transaction state machine, QR signing and moderation logic.
Database	MongoDB via the Motor async driver	Document store for users, catalog, and the full transaction lifecycle, each record keyed by a unique string ID.
Auth & Security	JWT (HS256, 7-day), bcrypt, TOTP 2FA, UTM-email regex gate	Verified-member sessions, hashed passwords, and MFA-gated admin moderation.
QR Verification	HMAC-SHA256 signed, 24-hour tokens (UTMB.<payload>.<sig>)	Tamper-proof, server-verified proof of physical handover and return.
Real-time	Server-Sent Events (SSE)	Push-only live updates for notifications, catalog and transaction changes, and chat.
Deployment	Render (Docker · uvicorn) + Vercel	FastAPI backend and React build wired through environment variables and a CORS allowlist.
Modeling & PM	Visual Paradigm, Figma, Jira, GitHub, Generative AI	UML modeling, UI/UX prototyping, sprint tracking, version control, and development acceleration.

1.5 Functional Requirements

The platform's functional capabilities are distributed across nine modules within three subsystems. The system shall:

Identity & User Management

- Register and authenticate users only with valid UTM email addresses, hashing every password with bcrypt.
- Provide secure, token-based password recovery.
- Maintain editable profiles and compute a dynamic 5-star Trust Score from completed transactions.
- Let users rate counterparties and report users or items for moderation.
- Present a real-time dashboard of items the user is lending and borrowing, with overdue alerts.

Resource Catalog & Discovery

- Let owners publish, edit, set visibility for, and remove their item listings.
- Enforce controlled category and condition metadata on every listing.
- Let borrowers search and filter the catalog by category, condition, college, and faculty, and sort by distance.

Transaction & Handover

- Submit borrow requests with date validation and an atomic Available → Pending item lock.
- Allow lender approval or rejection and request cancellation before handover.
- Generate and verify HMAC-SHA256 signed QR codes for handover and return.
- Log every transaction state change immutably and notify both parties in real time.
- Provide an MFA-gated admin moderation portal to review reports and remove items or suspend users.

2.0 APPLICATION DIAGRAM

2.1 Overall Use Case Diagram

Two actors drive the system. The UTM Student acts as both Lender and Borrower and interacts with all three subsystems, while the Administrator / Moderator operates the moderation surface of the Transaction & Handover subsystem. The diagram groups the platform's twenty-seven use cases by subsystem; each subsystem's detailed use-case diagram (in the SRS) shows the per-use-case actor associations.

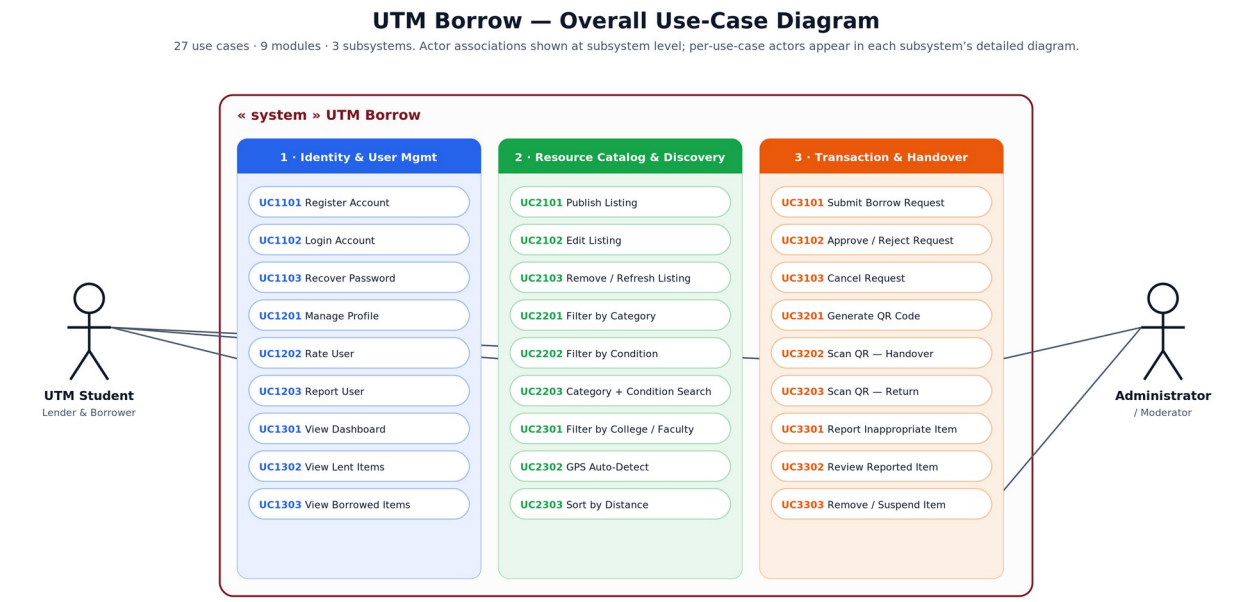


Figure 2.1 · Overall use-case diagram — 27 use cases across 9 modules and 3 subsystems.

2.2 Architecture Design

UTM Borrow follows a classic three-tier client–server architecture. The presentation tier is a React single-page application that renders every screen and talks to the server exclusively over HTTPS using Axios. The application tier is a FastAPI server exposing a REST API under /api; it handles authentication (JWT + bcrypt), input validation, the transaction state machine, QR signing and verification (HMAC-SHA256), and moderation logic. The data tier is a MongoDB database accessed through the asynchronous Motor driver, storing users, items, transactions, and supporting collections as documents. Alongside the request/response REST flow, a one-way Server-Sent Events channel pushes live updates to connected clients so the interface updates without polling.

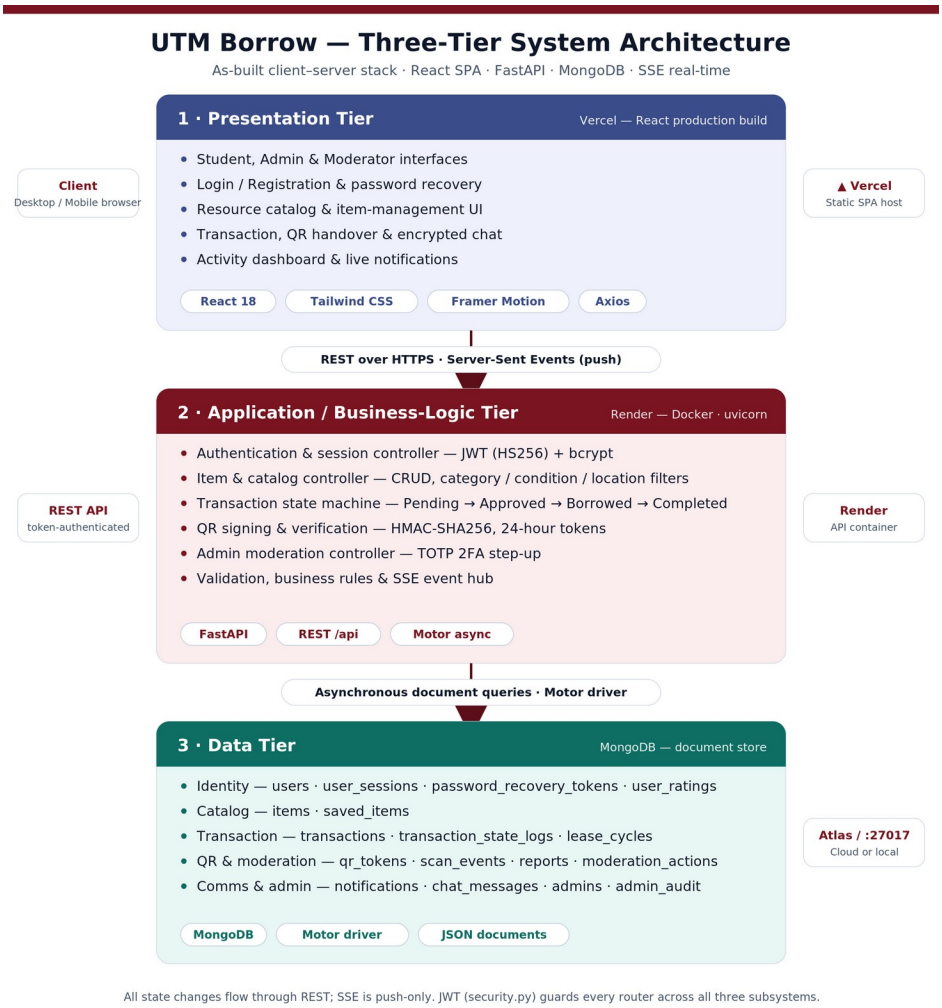


Figure 2.2 · As-built three-tier architecture (React SPA · FastAPI · MongoDB), deployed on Vercel and Render.

2.3 Technology Used and Integration

The three subsystems are built independently but reconnect into one running application at two integration points. On the backend, backend/server.py imports and mounts every subsystem's router into a single FastAPI app; on the frontend, frontend/src/App.js mounts every subsystem's pages into one route table. Cross-subsystem behaviour flows through three shared seams:

- **Identity → all subsystems.** security.py (JWT + bcrypt) guards every router, so a single verified session authorises actions across all three subsystems.
- **Resource → Transaction.** The items catalog published by Resource Catalog & Discovery is exactly what the Transaction & Handover flow locks, lends, and returns.
- **Transaction → Identity / Resource (real-time).** realtime.py and notifications.py push SSE events that the dashboard, listings, and notifications consume live.

In deployment, the React build is hosted on Vercel and the FastAPI server on Render (Docker, python:3.12-slim, uvicorn), wired together through environment variables and a CORS allowlist. All state-changing operations go through REST; SSE is push-only.

Completion Matrix — Status Legend

Each subsystem below reports its use cases in the completion-matrix format prescribed for this project. Development status is assessed against the delivered repository build: a use case is marked Fully Implemented only where a working REST endpoint, its MongoDB collection(s), and the corresponding React screen are all present and wired together. The four permitted status values are:

✔ Fully Implemented	⚠ Prototype (High-fidelity)	⚠ Prototype (Low-fidelity)	✖ Not Complete

3.0 IDENTITY & USER MANAGEMENT SUBSYSTEM

Subsystem owner: Ahmat Mahamat Mourdji Moustapha (A24CS4053) · Modules: Authentication · Profile & Trust Score · User Activity Dashboard

3.1 Completion Matrix

UC ID	Use Case	Development Status	Remarks (as-built evidence)	Figure
Module 1 — Authentication				
UC1101	Register Account	✔ Fully Implemented	UTM-email regex gate (@utm.my / @graduate.utm.my); bcrypt password hash; writes the users collection. POST /api/auth/register.	Fig 3.1
UC1102	Login Account	✔ Fully Implemented	JWT HS256 (7-day) session persisted in user_sessions. POST /api/auth/login; GET /api/auth/me.	Fig 3.1
UC1103	Recover Password	✔ Fully Implemented	Signed reset token in password_recovery_tokens. POST /forgot-password + /reset-password; SMTP-configurable (email simulated in dev build).	Fig 3.1
Module 2 — Profile & Trust Score				
UC1201	Manage Profile	✔ Fully Implemented	GET / PUT /api/profile; server-validated profile-picture upload; updates users.	Fig 3.2
UC1202	Rate User	✔ Fully Implemented	5-star post-transaction rating in user_ratings (unique index); recomputes users.trust_score. POST /api/ratings.	Fig 3.2
UC1203	Report User	✔ Fully Implemented	POST /api/moderation/reports/user → reports; routed to the admin moderation queue.	Fig 3.2
Module 3 — User Activity Dashboard				
UC1301	View Dashboard	✔ Fully Implemented	GET /api/dashboard aggregates active lends/borrows and overdue items; refreshed live via SSE.	Fig 3.3
UC1302	View Lent Items	✔ Fully Implemented	GET /api/transactions/lending; ActiveLoan and Urgent banners surface	Fig 3.3

UC ID	Use Case	Development Status	Remarks (as-built evidence)	Figure
			open commitments.	
UC1303	View Borrowed Items	✔ **Fully Implemented**	GET /api/transactions/borrowing with overdue alerts.	Fig 3.3

3.2 Figures

The screens below realise the Identity & User Management use cases. Each figure links to its React source on GitHub; insert the matching screenshot from the running application in the space provided.

Figure 3.1 — Authentication — Login / Register
UTM-email login and registration with password recovery (UC1101–UC1103).

⟦ paste screenshot of Authentication — Login / Register from the running app here ⟧

GitHub source: frontend/src/pages/identity/Login.js

Figure 3.2 — Profile & Trust Score
Editable profile, 5-star Trust Score, ratings and user reporting (UC1201–UC1203).

⟦ paste screenshot of Profile & Trust Score from the running app here ⟧

GitHub source: frontend/src/pages/identity/Profile.js

Figure 3.3 — User Activity Dashboard
Real-time view of items being lent and borrowed with overdue alerts (UC1301–UC1303).

⟦ paste screenshot of User Activity Dashboard from the running app here ⟧

GitHub source: frontend/src/pages/identity/Dashboard.js

Complete screen index — Identity & User Management

Fig.	Screen	Use case(s)	GitHub source
3.1	**Login**	UC1102	frontend/src/pages/identity/Login.js

Fig.	Screen	Use case(s)	GitHub source
3.1	**Register**	UC1101	frontend/src/pages/identity/Register.js
3.1	**Forgot / Reset Password**	UC1103	frontend/src/pages/identity/ForgotPassword.js
3.2	**Profile**	UC1201	frontend/src/pages/identity/Profile.js
3.2	**Public Profile**	UC1201	frontend/src/pages/identity/PublicProfile.js
3.2	**Reputation / Ratings**	UC1202, UC1203	frontend/src/pages/identity/Reputation.js
3.3	**Dashboard**	UC1301–UC1303	frontend/src/pages/identity/Dashboard.js
—	**Security & 2FA settings**	UC1102	frontend/src/pages/identity/SettingsSecurity.js
—	**Backend routers**	UC1101–UC1303	backend/routers/auth.py

4.0 RESOURCE CATALOG & DISCOVERY SUBSYSTEM

Subsystem owner: Muaz Ibne Ahmed (A23CS4062) · Modules: Item Listing & Management · Category & Condition Metadata · Location-Based Filter

4.1 Completion Matrix

UC ID	Use Case	Development Status	Remarks (as-built evidence)	Figure
Module 1 — Item Listing & Management				
UC2101	Publish Listing	✔ Fully Implemented	POST /api/items with server-side ENUM validation; writes the items collection. ItemForm screen.	Fig 4.1
UC2102	Edit Listing	✔ Fully Implemented	PUT /api/items/{id}; owner-guarded edit of an existing listing.	Fig 4.1
UC2103	Remove / Refresh Listing	✔ Fully Implemented	DELETE /api/items/{id}, PATCH /visibility, POST /refresh; Private/Removed items hidden from the catalog.	Fig 4.2
Module 2 — Category & Condition Metadata				
UC2201	Filter by Category	✔ Fully Implemented	GET /api/items?category; controlled category ENUM enforced on every write.	Fig 4.3
UC2202	Filter by Condition	✔ Fully Implemented	GET /api/items?condition; condition ENUM (New / Good / Fair).	Fig 4.3
UC2203	Combined Category + Condition Search	✔ Fully Implemented	GET /api/items?category&condition composite query.	Fig 4.3
Module 3 — Location-Based Filter				
UC2301	Filter by College / Faculty	✔ Fully Implemented	GET /api/items?college&faculty over controlled college and faculty lists.	Fig 4.3
UC2302	GPS Auto-Detect	✔ Fully Implemented	Browser geolocation combined with server-side UTM campus coordinate mapping; no third-party map service required.	Fig 4.3
UC2303	Sort by Distance	✔ Fully Implemented	Great-circle (haversine) distance over campus coordinates; distance_asc / distance_desc sorting.	Fig 4.3

4.2 Figures

The screens below realise the Resource Catalog & Discovery use cases. Each figure links to its React source on GitHub; insert the matching screenshot from the running application in the space provided.

Figure 4.1 — Add / Edit Listing (ItemForm)
Create and edit listings with category, condition, and college/faculty metadata (UC2101–UC2102).

〖 paste screenshot of Add / Edit Listing (ItemForm) from the running app here 〗

GitHub source: frontend/src/pages/resource/ItemForm.js

Figure 4.2 — My Listings (Lend)
Owner view to manage, hide, refresh, or remove listings (UC2103).

〖 paste screenshot of My Listings (Lend) from the running app here 〗

GitHub source: frontend/src/pages/resource/Lend.js

Figure 4.3 — Catalog & Filters (Home)
Category, condition, college/faculty filtering and distance sort (UC2201–UC2303).

〖 paste screenshot of Catalog & Filters (Home) from the running app here 〗

GitHub source: frontend/src/pages/resource/Home.js

Complete screen index — Resource Catalog & Discovery

Fig.	Screen	Use case(s)	GitHub source
4.3	**Home (catalog + filter panel)**	UC2201–UC2303	frontend/src/pages/resource/Home.js
4.2	**Lend (My Listings)**	UC2103	frontend/src/pages/resource/Lend.js
4.1	**ItemForm (Add / Edit)**	UC2101, UC2102	frontend/src/pages/resource/ItemForm.js
—	**ItemDetail**	UC2101	frontend/src/pages/resource/ItemDetail.js
—	**NearbyAll (distance discovery)**	UC2301–UC2303	frontend/src/pages/resource/NearbyAll.js
—	**PopularAll**	UC2201–UC2203	frontend/src/pages/resource/PopularAll.js
—	**Backend router**	UC2101–UC2303	backend/routers/items.py

5.0 TRANSACTION & HANDOVER SUBSYSTEM

Subsystem owner: Mohammed Mohsen Mohammed Alsakkaf (A23CS4026) · Modules: Request & Approval Workflow · QR Verification · Community Moderation & Reporting

5.1 Completion Matrix

UC ID	Use Case	Development Status	Remarks (as-built evidence)	Figure
Module 1 — Request & Approval Workflow				
UC3101	Submit Borrow Request	✔ Fully Implemented	POST /api/transactions; atomic Available → Pending item lock with date validation; writes transactions.	Fig 5.1
UC3102	Approve / Reject Request	✔ Fully Implemented	POST /{id}/approve | /reject; lender-guarded; every change written to transaction_state_logs.	Fig 5.1
UC3103	Cancel Request	✔ Fully Implemented	POST /{id}/cancel — borrower before handover, lender after approval; item restored to Available.	Fig 5.2
Module 2 — QR Verification				
UC3201	Generate QR Code	✔ Fully Implemented	HMAC-SHA256 signed token UTMB.<payload>.<sig>, 24-hour TTL in qr_tokens; auto-regenerates on expiry.	Fig 5.3
UC3202	Scan QR — Handover	✔ Fully Implemented	POST /api/scan (handover) → Approved → Borrowed atomically; writes scan_events and lease_cycles.	Fig 5.3
UC3203	Scan QR — Return	✔ Fully Implemented	POST /api/scan (return) → Borrowed → Completed; opens the post-return rating dialog.	Fig 5.3
Module 3 — Community Moderation & Reporting				
UC3301	Report Inappropriate Item	✔ Fully Implemented	POST /api/moderation/reports → reports; duplicate reports blocked; ReportModal component.	Fig 5.4
UC3302	Review Reported Item (Admin)	✔ Fully Implemented	GET /api/moderation/admin/reports via MFA-elevated adminApi; Senior_Moderator RBAC enforced.	Fig 5.5
UC3303	Remove / Suspend (Admin)	✔ Fully Implemented	POST /remove-item and /suspend-user; writes moderation_actions, user_suspensions, admin_audit.	Fig 5.5

5.2 Figures

The screens below realise the Transaction & Handover use cases — the accountability engine of UTM Borrow. Each figure links to its React source on GitHub; insert the matching screenshot from the running application in the space provided.

Figure 5.1 — Transaction Detail & Approval
Loan lifecycle with approve / reject / cancel controls and live state (UC3101–UC3103).

⟦ paste screenshot of Transaction Detail & Approval from the running app here ⟧

GitHub source: frontend/src/pages/transaction/TransactionDetail.js

Figure 5.3 — QR Scanner (Handover / Return)
Camera scan of HMAC-signed QR to advance handover and return (UC3201–UC3203).

⟦ paste screenshot of QR Scanner (Handover / Return) from the running app here ⟧

GitHub source: frontend/src/pages/transaction/Scanner.js

Figure 5.4 — Report an Item (Moderation)
Community reporting of inappropriate listings (UC3301).

⟦ paste screenshot of Report an Item (Moderation) from the running app here ⟧

GitHub source: frontend/src/pages/transaction/Moderation.js

Figure 5.5 — Admin Moderation Portal
MFA-gated review queue with remove-item and suspend-user actions (UC3302–UC3303).

⟦ paste screenshot of Admin Moderation Portal from the running app here ⟧

GitHub source: frontend/src/pages/admin/AdminReports.js

Complete screen index — Transaction & Handover

Fig.	Screen	Use case(s)	GitHub source
5.1	**TransactionDetail**	UC3101–UC3103	frontend/src/pages/transaction/TransactionDetail.js

Fig.	Screen	Use case(s)	GitHub source
—	**TransactionHistory**	UC3101–UC3103	frontend/src/pages/transaction/TransactionHistory.js
5.3	**Scanner (QR)**	UC3201–UC3203	frontend/src/pages/transaction/Scanner.js
5.4	**Moderation / ReportDetail**	UC3301	frontend/src/pages/transaction/Moderation.js
5.5	**Admin portal (Reports, Scan, Users)**	UC3302, UC3303	frontend/src/pages/admin/AdminReports.js
—	**Chat (encrypted)**	UC3301	frontend/src/pages/transaction/Chat.js
—	**Backend routers**	UC3101–UC3303	backend/routers/transactions.py

6.0 COMMUNITY ENGAGEMENT AND CONCLUSION

6.1 Community Engagement

UTM Borrow grew directly out of engagement with the campus community. During requirements gathering the team ran a questionnaire that reached students across multiple faculties and years of study and gathered twenty-nine responses. The findings validated every problem the platform now addresses: a majority of respondents admitted to repeatedly buying single-use academic items, most spent between RM50 and RM150 a year on them, and the strongest fear when lending was that an item would never be returned — with many relying only on chat screenshots as proof.

Three stakeholder groups shaped the design. Student borrowers needed a fast, searchable directory filtered by residential college and faculty, which became the Location-Based Filter. Student lenders needed confidence that items would come back, which became the verified identity gate, the Trust Score, and the signed-QR handover protocol. UTM administration and moderators needed a simple way to keep the community safe, which became the MFA-gated moderation portal. Each major feature is therefore traceable to a documented community need rather than assumption.

Figure 6.1 — Community Stakeholder Engagement
Photograph taken with the community stakeholder / representative during engagement.

⟦ paste the team & community-stakeholder photograph here ⟧

Insert a photograph captured during your community engagement session.

6.2 Conclusion

UTM Borrow was delivered as a working high-fidelity prototype that implements all twenty-seven use cases across nine modules and three subsystems on a genuine, deployed technology stack — a React single-page application, a FastAPI service, and a MongoDB database, hosted on Vercel and Render. The platform enforces real accountability: access is limited to verified UTM members, every loan is driven by an auditable state machine, and each physical handover and return is proven with a cryptographically signed QR code. In doing so it meets the course objective of building a high-fidelity prototype with diverse stakeholder involvement, and it advances UTM's sustainability agenda by turning a wasteful buy-and-discard habit into a managed circular-borrowing cycle.

Several enhancements remain as natural next steps: enabling production email delivery through a configured SMTP provider, packaging the responsive web app as an installable mobile application, integrating an optional e-wallet deposit for high-value items, and scaling the circular-sharing model to other Malaysian universities. These build on — rather than rework — the delivered system, which already stands as a complete, demonstrable platform.

Group Alpha · UTM Borrow · github.com/Alsakkaf-dev/UTMBorrow · Faculty of Computing, Universiti Teknologi Malaysia